// ZPWR-MIDI-FX //

zpwr-midi-fx - Reference Manual

v0.3.0

MODULE REFERENCE

note-stream MIDI effect + generator modules

2026-06-21

MenkeTechnologies

Contents

Plugin Architecture

JUCE MIDI-effects plugin --- the MIDI-domain companion to zpwr-fx, on the same shared patch-graph engine. Paid product.

Build & Position

Metric	Value	Notes
Language	C++	JUCE MIDI-effect plugin
Engine	LayeredEngine<PatchEngine<NoteStream>>	shared zpwr-patch-core graph instantiated on the note-event stream
Formats	VST3 · AU · CLAP · Standalone	CLAP via clap-juce-extensions; registers as a MIDI-effect category where the host supports it (Logic AU MIDI FX, CLAP note ports)
Targets	macOS arm64/x86_64 · Linux x86_64/aarch64 · Windows x64/arm64	cross-platform, cross-arch from day one (Windows builds VST3 + CLAP via scripts/build_win.ps1)
Build system	CMake	CMakeLists.txt at repo root
Modules	78 module types	registerMidiModules --- arp/chord/scale/euclid/transform/humanize/…; +1240+ audio/synth in the stack = 1300+ blocks
Domain	MIDI note stream	transforms notes before they reach an instrument
Status	stable	patch graph + full 78-module note-stream library, UI + factory presets

Architecture

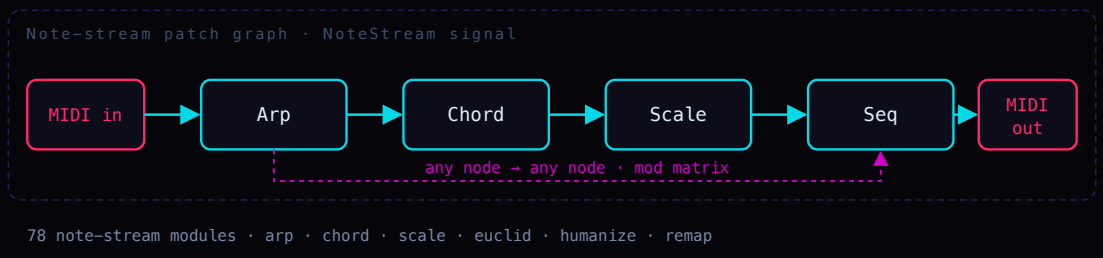


Figure 1: architecture diagram

The same zpwr-patch-core engine as zpwr-fx, instantiated on zmidi::NoteStream instead of float.

Not a fixed-slot rack: the same free-routed patch graph as zpwr-fx, instantiated on zmidi::NoteStream instead of float. Module types cover arpeggiation, chord generation, scale quantization, Euclidean and generative sequencing, velocity / timing humanize, and note remapping.

Catalog at a glance

78 blocks shipped by **zpwr-midi-fx** across **9 categories**, including 0 component-level analog-circuit models and 31 control / modulation (CV-badge) sources. This is the per-plugin subset of the shared zpwr-patch-core catalog.

Registry MIDI 78

Output jack Audio 0 MIDI 78 CV 0

By category (block count, largest first):

Sequencing	25	Control	5
Harmony	18	Voicing	4
Routing	9	Probability	1
Dynamics	8	Tuning	1
Modulation	7		

Reading this catalog (badge key)

An icon glyph precedes each block name; then its badges, then in: its input jacks and out: its output-jack type, then the description:

> [CAT] **category** - **AMP** Amp, **ANA** Analog, **CTL** Control, **DLY** Delay, **DST** Distortion, **DYN** Dynamics, **ENV** Envelope, **FLT** Filter, **FX** Effect, **HRM** Harmony, **MOD** Modulation, **OSC** Oscillator, **PIT** Pitch, **PRB** Probability, **REV** Reverb, **RTE** Routing, **SEQ** Sequencing, **SPC** Spectral, **STR** Stereo, **TUN** Tuning, **UTL** Utility, **VOI** Voicing.

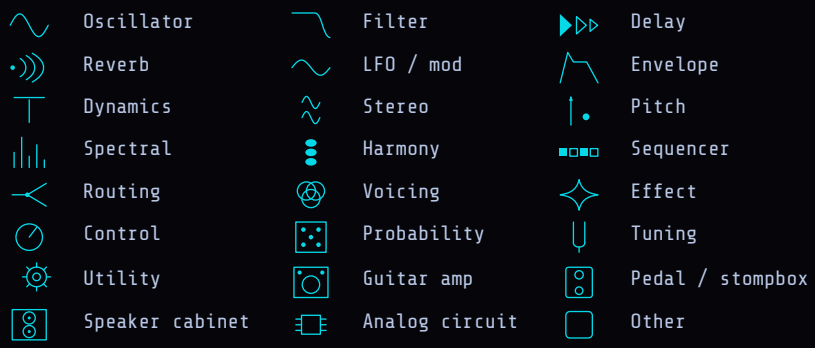
> [CV] a **control / modulation source** (outputs CV, not audio).

> [CIRCUIT] a **component-level analog-circuit model** (per-sample nodal / Newton solve).

> [FX] [SYNTH] [MIDI] which **plugin(s)** ship the block - zpwr-fx, zpwr-synth (gets the whole audio pack) and/or zpwr-midi-fx.

> **in:** the block's input jacks, and **out:** its output-jack signal type - out: Audio, out: MIDI (note stream) or out: CV (control / modulation).

> **icon:** the glyph before each block name shows its kind at a glance:



MIDI (zpwr-midi-fx) (78 blocks)

Control (5)

ATProc [CTL][CV][MIDI] in: Audio out: MIDI

Scales aftertouch, optionally converting it to a CC.

BendProc [CTL][CV][MIDI] in: Audio out: MIDI

Scales and/or inverts incoming pitch-bend.

CC2Note [CTL][CV][MIDI] in: Audio out: MIDI

A watched CC crossing a threshold triggers a note.

CCMap [CTL][CV][MIDI] in: Audio out: MIDI

Remaps a CC number and scales/offsets its value.

Note2CC [CTL][CV][MIDI] in: Audio out: MIDI

Emits a CC from each note (pitch or velocity).

Dynamics (8)

Accent [DYN][MIDI] in: Audio out: MIDI

Boosts velocity on every Nth note.

Humanize [DYN][MIDI] in: Audio out: MIDI

Adds random timing and velocity jitter for a human feel.

Ramp [DYN][MIDI] in: Audio out: MIDI

Velocity ramp over a run of notes, ascending or descending.

VelClip [DYN][MIDI] in: Audio out: MIDI

Clamps velocity between Min and Max.

VelCompress [DYN][MIDI] in: Audio out: MIDI

Velocity compressor: velocities above Threshold are pulled toward it by Ratio (1 = off, 8 = hard), narrowing the dynamic range smoothly instead of hard-clamping like VelClip.

VelCurve [DYN][MIDI] in: Audio out: MIDI

Reshapes velocity through a power curve.

VelInvert [DYN][MIDI] in: Audio out: MIDI

Flips the velocity scale around its midpoint (soft <-> loud) by Amount, so your accents become ghost notes and the ghosts become accents - a velocity mirror, blendable to taste.

Velocity [DYN][MIDI] in: Audio out: MIDI

Scales and offsets note velocity, with a modulation amount.

Harmony (18)

Bassline [HRM][MIDI] in: Audio out: MIDI

Follows the chord with a mono bass voice: the lowest held note dropped Octaves down, retriggered whenever the lowest note changes. Passes the original notes through and adds the bass beneath them.

Chord [HRM][MIDI] in: Audio out: MIDI

Turns each incoming note into a chord (165 types) with inversion, spread, octave doubling and strum.

ChordMem [HRM][MIDI] in: Audio out: MIDI

Each key plays a stored chord shape (semitone intervals from node config).

ChordProg [HRM][MIDI] in: Audio out: MIDI

Each note triggers the next chord in a stored root progression (node config).

ChordVoicing [HRM][MIDI] in: Audio out: MIDI

Re-voices each note into a jazz shape (open / shell / quartal / spread).

Counterpoint [HRM][MIDI] in: Audio out: MIDI

Adds a second voice in contrary motion - when the melody rises the counter voice falls and vice-versa - quantised to Root/Scale. Interval sets the starting separation.

Diatonic [HRM][MIDI] in: Audio out: MIDI

Transposes by scale DEGREES within Root/Scale (a third, a fifth…) instead of by raw semitones like Transpose - the shift always lands back in key. Mod Amt adds a modulatable degree offset.

DiatonicChord [HRM][MIDI] in: Audio out: MIDI

Auto-harmoniser: builds the in-key chord on every note by stacking diatonic thirds (triad, 7th…) along Root/Scale, so single-finger melodies become correct chords in the key.

Drone [HRM][MIDI] in: Audio out: MIDI

Doubles every note with a fixed pedal root.

Glissando [HRM][MIDI] in: Audio out: MIDI

Fills a stepwise run between consecutive notes.

Harmonize [HRM][MIDI] in: Audio out: MIDI

Adds up to three fixed-interval harmony voices to each note.

Invert [HRM][MIDI] in: Audio out: MIDI

Mirrors pitch around a pivot note.

MidiCluster [HRM][MIDI] in: Audio out: MIDI

Adds symmetric tone-cluster voices above and below each note.

MidiFold [HRM][MIDI] in: Audio out: MIDI

Folds out-of-range notes back inside a Low..High window.

NeoRiemann [HRM][MIDI] in: Audio out: MIDI

Neo-Riemannian triad transforms: builds a major/minor triad on each note and applies P (parallel), L (leading-tone) or R (relative) - the maximally-smooth chord moves of late-Romantic and film harmony, each sharing two common tones with the input.

Octave [HRM][MIDI] in: Audio out: MIDI

Doubles each note up and/or down by whole octaves.

Scale [HRM][MIDI] in: Audio out: MIDI

Quantizes incoming notes to the nearest degree of the chosen root and scale.

Transpose [HRM][MIDI] in: Audio out: MIDI

Shifts every note by a fixed semitone amount plus a modulatable offset.

Modulation (7)

Generative [MOD] [MIDI] in: Audio out: MIDI

Probabilistic melodic random walk synced to a rate.

MidiEnv [MOD] [MIDI] in: Audio out: MIDI

ADSR envelope triggered by the note gate, on the scalar control output.

MidiLFO [MOD] [MIDI] in: Audio out: MIDI

Low-frequency modulation source (sine/tri/saw/square) on the scalar control output.

MidiRandom [MOD] [MIDI] in: Audio out: MIDI

Generates random notes within a pitch range at a synced rate.

MidiSampleHold [MOD] [MIDI] in: Audio out: MIDI

Samples and holds the incoming control value on each note.

MidiSlew [MOD] [MIDI] in: Audio out: MIDI

Smooths (glides) the control value over a set time.

RandOctave [MOD] [MIDI] in: Audio out: MIDI

Randomly shifts notes by up to +/- Range octaves with a chance.

Probability (1)

Chance [PRB] [CV] [MIDI] in: Audio out: MIDI

Probabilistic gate: lets each note through with the set probability.

Routing (9)

Channel [RTE] [MIDI] in: Audio out: MIDI

Reassigns notes to a fixed MIDI channel.

FixedNote [RTE] [MIDI] in: Audio out: MIDI

Forces every note to a single fixed pitch (drum / trigger).

KeySwitch [RTE] [MIDI] in: Audio out: MIDI

Splits the keyboard at a note; one zone passes, the other is filtered.

Merge [RTE][MIDI] in: Audio out: MIDI

Passes both note-stream inputs through, merged into one stream.

MidiLatch [RTE][MIDI] in: Audio out: MIDI

Holds notes after release until the next note arrives.

NoteFilter [RTE][MIDI] in: Audio out: MIDI

Passes only notes inside a key and velocity window.

NoteLimit [RTE][MIDI] in: Audio out: MIDI

Polyphony cap: never lets more than Max notes sound at once, stealing the oldest (FIFO note-off) when a new note would exceed the limit - tames dense chords and runaway generators.

Split [RTE][MIDI] in: Audio out: MIDI

Keyboard split: notes below the split point transpose by Low, the rest by High.

Transformer [RTE][MIDI] in: Audio out: MIDI

Rule: notes within a range are transposed and velocity-scaled.

Sequencing (25)

Appoggiatura [SEQ][CV][MIDI] in: Audio out: MIDI

Leaning grace note: each struck note first sounds a long auxiliary (Interval away) that takes Time from the principal, which then sustains - the expressive long grace, unlike the quick Flam or Mordent.

Arp [SEQ][CV][MIDI] in: Audio out: MIDI

Arpeggiator: cycles held notes through a pattern at a synced rate with gate, octaves, swing and step count. Latch holds the chord so it keeps arpeggiating after the keys are released (the next key starts a new chord).

Cascade [SEQ][CV][MIDI] in: Audio out: MIDI

Each struck note fans out into a timed run of Steps notes climbing (or falling) the scale from it - an arpeggiated flourish fired per key, unlike Arp which cycles the whole held chord.

Echo [SEQ][CV][MIDI] in: Audio out: MIDI

MIDI delay: repeats notes at a synced time with velocity feedback decay.

▪▫▪▫ **Flam** [SEQ][CV][MIDI] in: Audio out: MIDI

Precedes every note with a quiet grace note, delaying the main hit so the grace leads it - the classic drum flam.

▪▫▪▫ **MidiBrianBrain** [SEQ][CV][MIDI] in: Audio out: MIDI

Brian's Brain MIDI sequencer: an 8x8 three-state grid (ready / firing / dying) evolves under Brian's Brain rules - a ready cell ignites only with exactly two firing neighbours, firing cells pass to dying, dying cells reset. Each clock step plays a note for every firing cell in the next column (row maps to a scale degree from Root/Scale); a full 8-step sweep advances one generation. The refractory state keeps it restless and sparkly where Game of Life settles. Density sets the random fill. The MIDI-note counterpart to the BrianBrain CV block.

▪▫▪▫ **MidiGameOfLife** [SEQ][CV][MIDI] in: Audio out: MIDI

Conway's Game of Life MIDI sequencer: an 8x8 cell grid evolves under the B3/S23 rule; each clock step reads the next column and plays a note for every live cell (row maps to a scale degree from Root/Scale), and a full 8-step sweep advances one generation. Gliders, blinkers and still-lifes become ever-shifting melodic and harmonic patterns. Density sets the random fill when the grid (re)seeds. The MIDI-note counterpart to the GameOfLife CV block.

▪▫▪▫ **MidiLangton** [SEQ][CV][MIDI] in: Audio out: MIDI

Langton's Ant MIDI melody: a single turmite walks an 8x8 grid - turning right on a white cell, left on a black one, flipping each cell as it leaves, then stepping forward. Each clock advances the ant Steps cells and plays one note for its current row (mapped to a scale degree from Root/Scale), so the melody scribbles chaotically then locks into the famous periodic `highway'. Emergent order from one trivial rule - a single moving agent, not a population.

▪▫▪▫ **MidiQuantize** [SEQ][CV][MIDI] in: Audio out: MIDI

Snaps note starts to a synced grid by a strength amount.

▪▫▪▫ **MidiTranceGate** [SEQ][CV][MIDI] in: Audio out: MIDI

Chops the held notes with a 16-step on/off Pattern at a synced Rate: every `on' step retriggers all held notes, gated to a fraction of the step. The rhythmic gate behind the trance sound - pattern-driven, where Roll is a fixed pulse.

▪▫▪▫ **MidiTuring** [SEQ][CV][MIDI] in: Audio out: MIDI

Turing-machine sequencer (Music Thing style): a circular shift register clocks once per step and the held notes fire when the read bit is 1. Change is the probability the recycled bit flips - 0 locks an evolving loop, 1 fully randomises, between = slowly mutating melodies.

▪▫▪▫ **Mordent** [SEQ][CV][MIDI] in: Audio out: MIDI

One-shot ornament: each struck note plays principal -> auxiliary -> principal at the attack, then sustains, the single-flick counterpart to the continuous Trill.

▪▫▪▫ **NoteLength** [SEQ][CV][MIDI] in: Audio out: MIDI

Forces a fixed note duration (gate length).

▪▫▪▫ **NoteOffDelay** [SEQ][CV][MIDI] in: Audio out: MIDI

Extends each note's gate by delaying its note-off.

▪▫▪▫ **PatternGate** [SEQ][CV][MIDI] in: Audio out: MIDI

Gates note-ons through a 16-step on/off pattern (bitmask).

▪▫▪▫ **Polymeter** [SEQ][CV][MIDI] in: Audio out: MIDI

Two step lanes of different lengths (LenA, LenB) clock together over the held notes, phasing against each other so the combined pattern only repeats after lcm(LenA,LenB) steps - instant polymetric movement from one chord.

▪▫▪▫ **Ricochet** [SEQ][CV][MIDI] in: Audio out: MIDI

Accelerating bouncing retriggers with decaying velocity.

▪▫▪▫ **Roll** [SEQ][CV][MIDI] in: Audio out: MIDI

Drum roll / buzz: retriggers every held note at Rate for as long as it is held, each hit gated to a fraction of the interval.

▪▫▪▫ **SeqEuclid** [SEQ][CV][MIDI] in: Audio out: MIDI

Euclidean rhythm gate: spreads Pulses evenly across Steps (rotatable) at a synced rate.

▪▫▪▫ **SeqRatchet** [SEQ][CV][MIDI] in: Audio out: MIDI

Retriggers each note Count times within a synced step.

▪▫▪▫ **StepSeqMidi** [SEQ][CV][MIDI] in: Audio out: MIDI

Melodic step sequencer - semitone offsets (node config) clock the held root.

▪▫▪▫ **Strum** [SEQ][CV][MIDI] in: Audio out: MIDI

Spreads a chord's notes over time, up or down, like a guitar strum.

▪▫▪▫ **Swing** [SEQ][CV][MIDI] in: Audio out: MIDI

Pushes off-beat eighth-notes later for a swing/shuffle feel.

▪▫▪▫ **Trill** [SEQ][CV][MIDI] in: Audio out: MIDI

Keyboard trill: while a note is held, alternates rapidly between it and the note Interval semitones above, at Rate.

Turn [SEQ][CV][MIDI] in: Audio out: MIDI

Gruppetto turn: a four-step ornament at the attack - upper auxiliary, principal, lower auxiliary, then the principal held. Time sets each step, Interval the neighbour distance.

Tuning (1)

Microtuner [TUN][MIDI] in: Audio out: MIDI

Stretched-octave microtuning via per-note pitch detune.

Voicing (4)

MidiUnison [VOI][MIDI] in: Audio out: MIDI

Doubles each note into multiple voices, optionally spread across channels.

Mono [VOI][MIDI] in: Audio out: MIDI

Monophonic note priority (last / lowest / highest).

MPEConvert [VOI][MIDI] in: Audio out: MIDI

Assigns each note its own rotating channel for per-note MPE expression.

Sustain [VOI][MIDI] in: Audio out: MIDI

Sustain-pedal simulation: while Hold is engaged, note-offs are captured so the notes keep sounding; when Hold drops, every held note releases at once. Hold is a parameter, so a CC, an LFO or automation can play the pedal.

Index (78 blocks, alphabetical)